IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 14

Project Sprint 4: Polish & Testing

Bug Fixes, UI Polish, Testing, dan Performance

Program Studi Teknik Informatika
Institut Teknologi Sumatera
Tahun Akademik Genap 2025/2026

📊 SPRINT PROGRESS - ALMOST THERE!

W11	Sprint 1: Planning & Setup	✓
W12	Sprint 2: Core Features	✓
W13	Sprint 3: Advanced Features	✓
W14	Sprint 4: Polish & Testing	◀ NOW
W15	Sprint 5: Final Preparation	
W16	UAS: Final Demo Day	🎯

🎯 Sprint 4 Goal: Polish UI, fix bugs, add tests - prepare for demo!

OUTLINE PERTEMUAN

01
Sprint 3 Review
Checkpoint dan feedback

02
Bug Fixing
Systematic debugging approach

03
UI Polish
Consistency, spacing, animations

04
Testing Strategy
Unit tests, UI tests, coverage

05
Performance
Optimization dan profiling

06
Sprint 4 Tasks
Work session dan deliverables

BAGIAN 1

BUG FIXING

Systematic Debugging Approach

BUG TRACKING DENGAN GITHUB ISSUES

```
## Bug Report Template

**Description:**
Short description of the bug

**Steps to Reproduce:**
1. Go to '...'
2. Click on '...'
3. See error

**Expected Behavior:**
What should happen

**Actual Behavior:**
What actually happens

**Screenshots:**
[If applicable]

**Device/Environment:**
- Device: Pixel 7 / iPhone 14
- OS: Android 14 / iOS 17
```

💡 **Labels untuk Issues:**

🔴 bug 🟡 enhancement 🟢 good first issue 🔵 help wanted ⚪ documentation

Priority: P0 (critical) → P1 (high) → P2 (medium) → P3 (low)

DEBUGGING STRATEGIES

Reproduce First

Pastikan bisa reproduce bug consistently sebelum fix

Isolate the Problem

Narrow down: screen mana? function mana? line mana?

Check Logs

Napier logs, Logcat, stack traces - read carefully!

Use Debugger

Breakpoints, step through, inspect variables

Rubber Duck

Explain problem out loud - often reveals solution

Git Bisect

Find which commit introduced the bug

COMMON BUGS & FIXES

❌ Crash on rotation

✓ Use ViewModel for state, handle configuration changes

❌ Memory leak

✓ Cancel coroutines in onCleared(), use weak references

❌ UI not updating

✓ Ensure StateFlow collected, check recomposition

❌ Network error not shown

✓ Proper try-catch, update UI state on error

❌ Back button issues

✓ Handle popBackStack correctly, check navigation

❌ Data not persisted

✓ Verify database transaction, check suspend context

BAGIAN 2

UI POLISH

Consistency, Spacing, dan Details

UI POLISH CHECKLIST

☐ **Typography**	Consistent fonts, sizes, weights across app
☐ **Spacing**	Consistent padding (8dp grid), margins, gaps
☐ **Colors**	Use theme colors, proper contrast, accessibility
☐ **Icons**	Consistent style (filled/outlined), proper sizes
☐ **Loading States**	Show progress indicators, skeleton screens
☐ **Error States**	Friendly messages, retry options, illustrations
☐ **Empty States**	Helpful messages, call-to-action buttons

SPACING SYSTEM

```
// Define consistent spacing values
object Spacing {
    val xs = 4.dp
    val sm = 8.dp
    val md = 16.dp
    val lg = 24.dp
    val xl = 32.dp
}

// Use throughout app
Column(
    modifier = Modifier.padding(Spacing.md),
    verticalArrangement = Arrangement.spacedBy(Spacing.sm)
) {
    Text("Title", style = MaterialTheme.typography.headlineMedium)
    Spacer(modifier = Modifier.height(Spacing.sm))
    Text("Description", style = MaterialTheme.typography.bodyMedium)
}
```

💡 **8dp Grid System:**

Semua spacing dan sizing kelipatan 8dp: 8, 16, 24, 32, 40, 48...
Ini membuat UI terlihat lebih organized dan consistent.

HANDLE EDGE CASES

Long Text

```
Text(
    text = item.title,
    maxLines = 2,
    overflow = TextOverflow.Ellipsis,
    style = MaterialTheme.typography.titleMedium
)
```

Image Placeholder

```
AsyncImage(
    model = item.imageUrl,
    contentDescription = item.title,
    placeholder = painterResource(R.drawable.placeholder),
    error = painterResource(R.drawable.error),
    contentScale = ContentScale.Crop
)
```

Edge cases to test:

• Empty list • Very long text • Missing images • Slow network • No network • Large data sets

BAGIAN 3

TESTING STRATEGY

Unit Tests, UI Tests, dan Coverage

TESTING GOALS UNTUK PROJECT

Repository Tests	**5+**	CRUD operations, error handling
ViewModel Tests	**4+**	State changes, user actions
Flow Tests	**2+**	Data streams, transformations
UI Tests	**3+**	Critical user journeys
Coverage Goal	**50%+**	Minimum coverage target

APA YANG HARUS DI-TEST?

✓ DO TEST:

- Business logic (calculations, validation)
- State transformations
- Data mapping (DTO → Domain)
- Error handling
- Critical user flows
- Edge cases (empty, null, max)

✗ DON'T TEST:

- Framework code (Compose, Ktor)
- Simple getters/setters
- Third-party libraries
- UI implementation details
- Platform-specific code (mostly)
- One-liner functions

💡 Testing Priority:

1. Repository (data correctness) → 2. ViewModel (state management) → 3. UI (user journeys)

REPOSITORY TEST EXAMPLE

```
class ItemRepositoryTest {
    private lateinit var repository: ItemRepository
    private lateinit var database: TestDatabase

    @BeforeTest
    fun setup() {
        database = createTestDatabase()
        repository = ItemRepositoryImpl(database)
    }

    @Test
    fun `insert item should add to database`() = runTest {
        // Arrange
        val item = Item(title = "Test", description = "Desc")

        // Act
        val id = repository.insertItem(item)

        // Assert
        val result = repository.getItemById(id).first()
        assertNotNull(result)
        assertEquals("Test", result.title)
    }

    @Test
    fun `delete item should remove from database`() = runTest {
        val id = repository.insertItem(Item(title = "ToDelete", description = ""))
        repository.deleteItem(id)

        val result = repository.getItemById(id).first()
        assertNull(result)
    }
}
```

VIEWMODEL TEST EXAMPLE

```
class HomeViewModelTest {
    private lateinit var viewModel: HomeViewModel
    private lateinit var repository: FakeItemRepository

    @BeforeTest
    fun setup() {
        repository = FakeItemRepository()
        viewModel = HomeViewModel(repository)
    }

    @Test
    fun `initial state should be Loading`() {
        assertEquals(HomeUiState.Loading, viewModel.uiState.value)
    }

    @Test
    fun `when items loaded should show Success`() = runTest {
        // Arrange
        repository.emit(listOf(Item(id = 1, title = "Test", description = "")))

        // Act & Assert
        viewModel.uiState.test {
            skipItems(1) // Skip Loading
            val state = awaitItem()
            assertTrue(state is HomeUiState.Success)
            assertEquals(1, (state as HomeUiState.Success).items.size)
        }
    }

    @Test
    fun `when empty should show Empty state`() = runTest {
        repository.emit(emptyList())
        viewModel.uiState.test {
            skipItems(1)
            assertEquals(HomeUiState.Empty, awaitItem())
        }
    }
}
```

UI TEST EXAMPLE

```
class HomeScreenTest {
    @get:Rule
    val composeTestRule = createComposeRule()

    @Test
    fun `empty state shows empty message`() {
        composeTestRule.setContent {
            HomeScreen(uiState = HomeUiState.Empty)
        }

        composeTestRule
            .onNodeWithText("No items yet")
            .assertIsDisplayed()

        composeTestRule
            .onNodeWithText("Add your first item")
            .assertIsDisplayed()
    }

    @Test
    fun `clicking add button navigates to add screen`() {
        var navigated = false

        composeTestRule.setContent {
            HomeScreen(
                uiState = HomeUiState.Success(emptyList()),
                onNavigateToAdd = { navigated = true }
            )
        }

        composeTestRule
            .onNodeWithContentDescription("Add item")
            .performClick()

        assertTrue(navigated)
    }
}
```

RUNNING TESTS & COVERAGE

```
# Run all tests
./gradlew test

# Run with coverage (Kover)
./gradlew koverHtmlReport

# Coverage report location
build/reports/kover/html/index.html
```

📊 **Coverage Report akan menunjukkan:**

- Line coverage: % kode yang dieksekusi oleh tests
- Branch coverage: % if/when branches yang di-test
- Class coverage: % class yang di-test

🎯 Target: 50% overall, 70%+ untuk critical code (Repository, ViewModel)

BAGIAN 4

PERFORMANCE

Optimization dan Profiling

COMPOSE PERFORMANCE TIPS

✓ Use keys in LazyColumn

```
items(list, key = { it.id }) { ... }
```

✓ Remember expensive operations

```
val sorted = remember(list) { list.sortedBy { it.name } }
```

✓ Use derivedStateOf

```
val isValid by derivedStateOf { name.isNotBlank() && email.isValid() }
```

✓ Avoid allocations in lambdas

```
onClick = onItemClick  // NOT: onClick = { onItemClick() }
```

⚠ Don't optimize prematurely! Measure first, optimize where needed.

BAGIAN 5

SPRINT 4 TASKS

Work Session dan Deliverables

SPRINT 4 GOALS

P0	**Fix All Known Bugs** No crash, no broken features
P0	**UI Polish** Consistent spacing, typography, colors
P0	**Unit Tests** Repository + ViewModel tests (10+ total)
P1	**UI Tests** Critical user journeys (3+)
P1	**Edge Cases** Empty states, errors, loading handled
P2	**Performance** No visible lag or jank

DELIVERABLES SPRINT 4

Bobot: 5% | Deadline: Sebelum Pertemuan 15

📋 Deliverables:

☐ All known bugs fixed (no crashes, all features work)

☐ UI polished (consistent design, proper states)

☐ 10+ unit tests (Repository + ViewModel)

☐ 3+ UI tests (critical flows)

☐ 50%+ code coverage

☐ README updated dengan test instructions

📤 Submission:

- GitHub repo dengan all fixes
- Video demo (2 min): show polished UI, run tests
- Coverage report screenshot in README

RUBRIK PENILAIAN SPRINT 4

Komponen	Bobot	Kriteria
Bug Fixes	25%	No crashes, all features functional
UI Polish	25%	Consistent, proper states, edge cases
Unit Tests	25%	10+ tests, meaningful assertions
UI Tests	15%	3+ tests, critical flows covered
Coverage	10%	50%+ coverage achieved

🌟 **Bonus: 70%+ coverage (+10%)**

⚠️ **Late: -10%/day**

🛠️ WORK SESSION

1	10 min	List all known bugs, prioritize P0 first
2	15 min	Fix critical bugs (crashes, broken features)
3	20 min	Write unit tests for Repository/ViewModel
4	10 min	UI polish - pick 1-2 screens to improve
5	5 min	Commit, push, check CI

🎯 Target hari ini:

- All P0 bugs identified and assigned
- At least 3 tests written and passing
- 1 screen fully polished

PREVIEW: SPRINT 5 & UAS

Sprint 5: Final Prep

- Final bug fixes
- Demo preparation
- Presentation slides
- Build signed APK
- Practice demo flow

UAS: Demo Day 🎉

- 10-15 min per team
- Live demo on device
- Q&A from panel
- Show all features
- Explain architecture

📊 **Final Grade Breakdown:**

Sprint 1-5: 5% × 5 = 25%
UAS Demo: 35%
Total Project: 60% of final grade

START PREPARING YOUR DEMO

✓ Practice the flow

Know exactly what to show, in what order

✓ Prepare test data

Have realistic data ready, not "test123"

✓ Handle failures gracefully

Know what to do if something breaks

✓ Highlight key features

Don't just click around, explain WHY

✓ Time yourself

Stay within 10-15 minutes

✓ Have backup plan

Screenshots/video if device fails

RESOURCES

Docs

Compose Testing — developer.android.com/jetpack/compose/testing

Docs

MockK — mockk.io

Docs

Turbine (Flow testing) — github.com/cashapp/turbine

Guide

UI Testing Best Practices — developer.android.com/training/testing/ui-testing

SESI TANYA JAWAB

CHECKLIST SEBELUM PULANG

- ☐ Bug list created dan prioritized
- ☐ Critical bugs (P0) fixed atau in progress
- ☐ At least 3 tests written
- ☐ 1 screen polished
- ☐ Code pushed ke repository
- ☐ CI passing
- ☐ Paham deliverables Sprint 4

2 Weeks to Demo Day! 

Sprint 4: Polish & Testing ← You are here
Sprint 5: Final Preparation
UAS: Demo Day 🎉

The home stretch! 🏃‍♂️

Polish Makes Perfect! ✨

"Quality is not an act,
it is a habit."

- Aristotle

See you next week for Sprint 5! 🎯

Terima Kasih

Happy Testing! 🧪

FINAL COUNTDOWN

Week 14: Sprint 4 - Polish & Testing ✓
Week 15: Sprint 5 - Final Preparation
Week 16: UAS - DEMO DAY! 🎉

You've got this! 💪